

ACTIVIDAD 2

PROGRAMACIÓN ORIENTADA A OBJETOS (POO)

Universidad Nacional de Colombia

Estudiante: Kevin David Aristizabal Giraldo

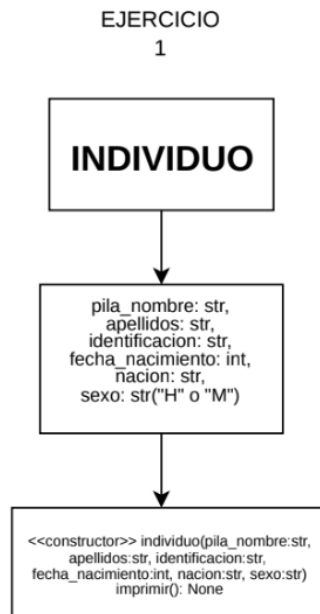
Docente: Walter Hugo Arboleda Mazo

INTRODUCCIÓN:

Documento que pretende recopilar las soluciones del libro de programación orientada a objetos de Leonardo Bermón Angarita, de la actividad 2, las cuales comprenden los ejercicios 2.1, 2.2, 2.3, 2.4, 2.5.

Ejercicio #2.1

Diagrama de clase:



Código fuente:

```
class Individuo:
    def __init__(self, pila_nombre: str, apellidos: str, identificacion: str,
                  fecha_nacimiento: int, nacion: str, sexo: str):
        if sexo not in ("H", "M"):
            raise ValueError("El género debe ser 'H' o 'M'.")
        self.pila_nombre = pila_nombre
        self.apellidos = apellidos
        self.identificacion = identificacion
        self.fecha_nacimiento = fecha_nacimiento
        self.nacion = nacion
        self.sexo = sexo

    def mostrar(self):
        print(f"Nombre = {self.pila_nombre}")
        print(f"Apellidos = {self.apellidos}")
        print(f"Número de documento de identidad = {self.identificacion}")
        print(f"Año de nacimiento = {self.fecha_nacimiento}")
```

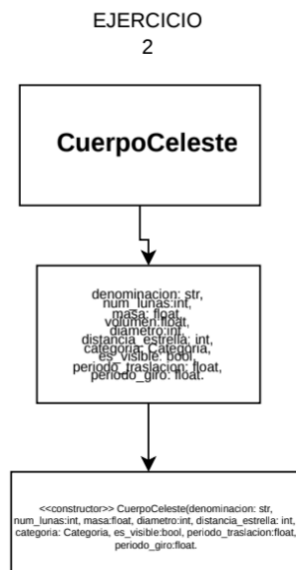
```
print(f"País de nacimiento = {self.nacion}")
print(f"Género = {self.sexo}")
```

```
def ejecutar():
    p1 = Individuo("Pedro", "Pérez", "1053121010", 1998, "Colombia", "H")
    p2 = Individuo("Luis", "León", "1053223344", 2001, "Colombia", "H")
    p1.mostrar()
    print()
    p2.mostrar()
```

```
if __name__ == "__main__":
    ejecutar()
```

Ejercicio #2.2

Diagrama de clase:



Código fuente:

```
from enum import Enum

class CategoriaCuerpo(Enum):
    GASEOSO = "GASEOSO"
    ROCOSO = "ROCOSO"
    ENANO = "ENANO"
```

```

class CuerpoCeleste:
    # 1 UA = 149 597 870 km. Se considera lejano un cuerpo más allá de 3,4 UA.
    DISTANCIA_UA_KM = 149_597_870
    UMBRAL_LEJANO_UA = 3.4

    def __init__(self, denominacion: str, num_lunas: int, masa: float,
                  volumen: float, diametro: int, distancia_estrella: int,
                  categoria: CategoriaCuerpo, es_visible: bool,
                  periodo_traslacion: float, periodo_giro: float):
        self.denominacion = denominacion
        self.num_lunas = num_lunas
        self.masa = masa                # kg
        self.volumen = volumen          # km^3
        self.diametro = diametro        # km
        self.distancia_estrella = distancia_estrella # km
        self.categoria = categoria
        self.es_visible = es_visible
        self.periodo_traslacion = periodo_traslacion # años
        self.periodo_giro = periodo_giro            # días

    def mostrar(self):
        print(f"Denominación del cuerpo = {self.denominacion}")
        print(f"Cantidad de lunas = {self.num_lunas}")
        print(f"Masa del cuerpo = {self.masa}")
        print(f"Volumen del cuerpo = {self.volumen}")
        print(f"Diámetro del cuerpo = {self.diametro}")
        print(f"Distancia a la estrella = {self.distancia_estrella}")
        print(f"Categoría del cuerpo = {self.categoria.name}")
        print(f"Es visible = {str(self.es_visible).lower()}")
        print(f"Periodo de traslación (años) = {self.periodo_traslacion}")
        print(f"Periodo de giro (días) = {self.periodo_giro}")
        print(f"Densidad del cuerpo = {self.obtener_densidad()}")
        print(f"Es cuerpo lejano = {str(self.es_cuerpo_lejano()).lower()}")

    def obtener_densidad(self):
        return self.masa / self.volumen

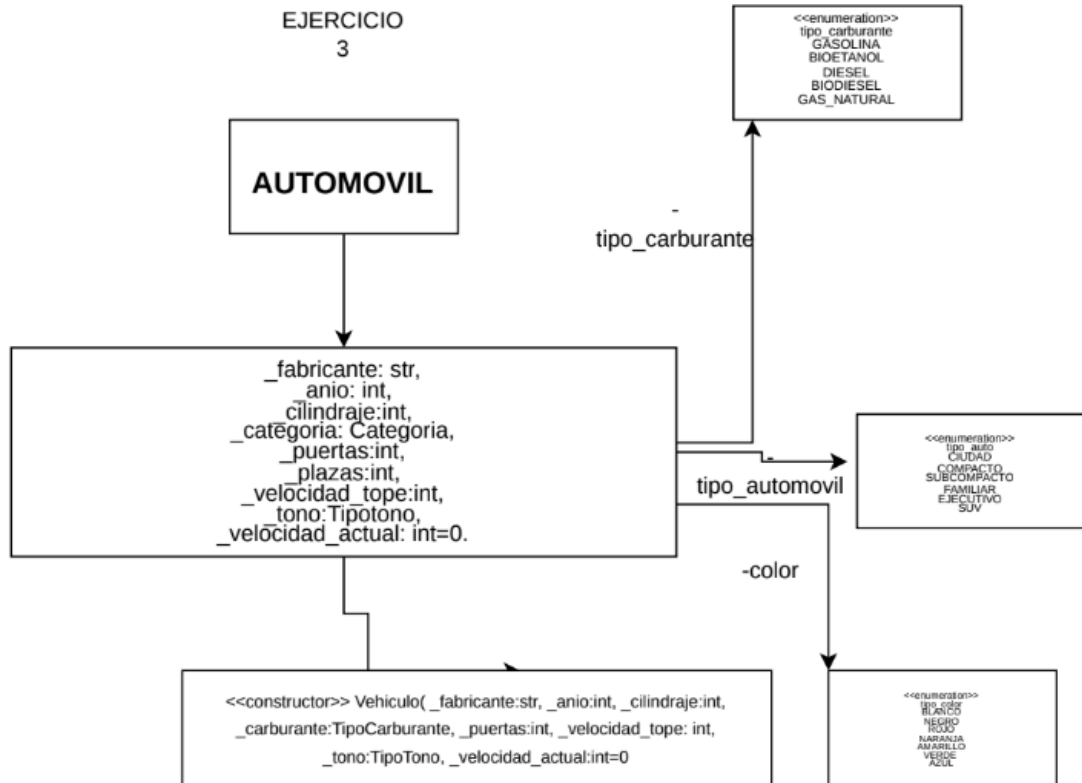
    def es_cuerpo_lejano(self):
        umbral = self.UMBRAL_LEJANO_UA * self.DISTANCIA_UA_KM
        return self.distancia_estrella > umbral

    def ejecutar():
        tierra = CuerpoCeleste("Tierra", 1, 5.9736e24, 1.08321e12, 12742, 150000000,
                                CategoriaCuerpo.ROCOSO, True, 1.0, 1.0)
        jupiter = CuerpoCeleste("Júpiter", 79, 1.899e27, 1.4313e15, 139820, 750000000,
                                 CategoriaCuerpo.GASEOSO, True, 11.86, 0.41)
        tierra.mostrar()
        print()
        jupiter.mostrar()

```

```
if __name__ == "__main__":
    ejecutar()
```

Ejercicio # 2.3



Código fuente:

```
from enum import Enum
```

```
class ClaseCarburante(Enum):
    GASOLINA = "GASOLINA"
    BIOETANOL = "BIOETANOL"
    DIESEL = "DIESEL"
    BIODIESEL = "BIODIESEL"
    GAS_NATURAL = "GAS_NATURAL"
```

```
class CategoriaVehiculo(Enum):
    URBANO = "URBANO"
    SUBCOMPACTO = "SUBCOMPACTO"
    COMPACTO = "COMPACTO"
    FAMILIAR = "FAMILIAR"
    EJECUTIVO = "EJECUTIVO"
    TODO_TERRENO = "TODO_TERRENO"
```

```
class TonoPintura(Enum):
    BLANCO = "BLANCO"
```

```
NEGRO = "NEGRO"
ROJO = "ROJO"
NARANJA = "NARANJA"
AMARILLO = "AMARILLO"
VERDE = "VERDE"
AZUL = "AZUL"
VIOLETA = "VIOLETA"
```

```
class Vehiculo:
```

```
    def __init__(self, fabricante: str, anio: int, cilindraje: int,
                  carburante: ClaseCarburante, categoria: CategoriaVehiculo,
                  puertas: int, plazas: int,
                  velocidad_tope: int, tono: TonoPintura):
        self._fabricante = fabricante
        self._anio = anio
        self._cilindraje = cilindraje
        self._carburante = carburante
        self._categoria = categoria
        self._puertas = puertas
        self._plazas = plazas
        self._velocidad_tope = velocidad_tope
        self._tono = tono
        self._velocidad_actual = 0
```

```
    @property
    def fabricante(self):
        return self._fabricante
```

```
    @fabricante.setter
    def fabricante(self, fabricante: str):
        self._fabricante = fabricante
```

```
    @property
    def anio(self):
        return self._anio
```

```
    @anio.setter
    def anio(self, anio: int):
        self._anio = anio
```

```
    @property
    def cilindraje(self):
        return self._cilindraje
```

```
    @cilindraje.setter
    def cilindraje(self, cilindraje: int):
        self._cilindraje = cilindraje
```

```
    @property
    def carburante(self):
        return self._carburante
```

```
    @carburante.setter
    def carburante(self, carburante: ClaseCarburante):
        self._carburante = carburante
```

```
    @property
    def categoria(self):
        return self._categoria
```

```
    @categoria.setter
    def categoria(self, categoria: CategoriaVehiculo):
        self._categoria = categoria
```

```
    @property
```

```

def puertas(self):
    return self._puertas

@puertas.setter
def puertas(self, puertas: int):
    self._puertas = puertas

@property
def plazas(self):
    return self._plazas

@plazas.setter
def plazas(self, plazas: int):
    self._plazas = plazas

@property
def velocidad_tope(self):
    return self._velocidad_tope

@velocidad_tope.setter
def velocidad_tope(self, velocidad_tope: int):
    self._velocidad_tope = velocidad_tope

@property
def tono(self):
    return self._tono

@tono.setter
def tono(self, tono: TonoPintura):
    self._tono = tono

@property
def velocidad_actual(self):
    return self._velocidad_actual

@velocidad_actual.setter
def velocidad_actual(self, velocidad_actual: int):
    self._velocidad_actual = velocidad_actual

def incrementar_velocidad(self, delta_velocidad: int):
    candidata = self._velocidad_actual + delta_velocidad
    if candidata > self._velocidad_tope:
        print("No se puede superar la velocidad máxima.")
        self._velocidad_actual = self._velocidad_tope
    else:
        self._velocidad_actual = candidata

def reducir_velocidad(self, delta_velocidad: int):
    candidata = self._velocidad_actual - delta_velocidad
    if candidata < 0:
        print("No se puede decrementar a una velocidad negativa.")
    else:
        self._velocidad_actual = candidata

def detener(self):
    self._velocidad_actual = 0

def estimar_tiempo_recorrido(self, distancia: float):
    """Tiempo en horas para recorrer 'distancia' (km) a la velocidad actual."""
    if self._velocidad_actual <= 0:
        raise ValueError("El vehículo está detenido; no se puede calcular el tiempo.")
    return distancia / self._velocidad_actual

def mostrar(self):
    print(f'Marca = {self._fabricante}')
    print(f'Modelo = {self._anio}')

```

```

print(f"Motor = {self._cilindraje}")
print(f"Tipo de combustible = {self._carburante.name}")
print(f"Tipo de automóvil = {self._categoria.name}")
print(f"Número de puertas = {self._puertas}")
print(f"Cantidad de asientos = {self._plazas}")
print(f"Velocidad máxima = {self._velocidad_tope}")
print(f"Color = {self._tono.name}")
print(f"Velocidad actual = {self._velocidad_actual}")

```

```

def ejecutar():
    auto1 = Vehiculo("Ford", 2018, 3, ClaseCarburante.DIESEL,
        CategoriaVehiculo.EJECUTIVO, 5, 6, 250, TonoPintura.NEGRO)
    auto1.mostrar()
    auto1.incrementar_velocidad(100)
    print(f"Velocidad actual = {auto1.velocidad_actual}")
    auto1.incrementar_velocidad(20)
    print(f"Velocidad actual = {auto1.velocidad_actual}")
    auto1.reducir_velocidad(50)
    print(f"Velocidad actual = {auto1.velocidad_actual}")
    print(f"Tiempo de llegada para 140 km = {auto1.estimated_time(140)} horas")
    auto1.detener()
    print(f"Velocidad actual = {auto1.velocidad_actual}")
    auto1.reducir_velocidad(10)

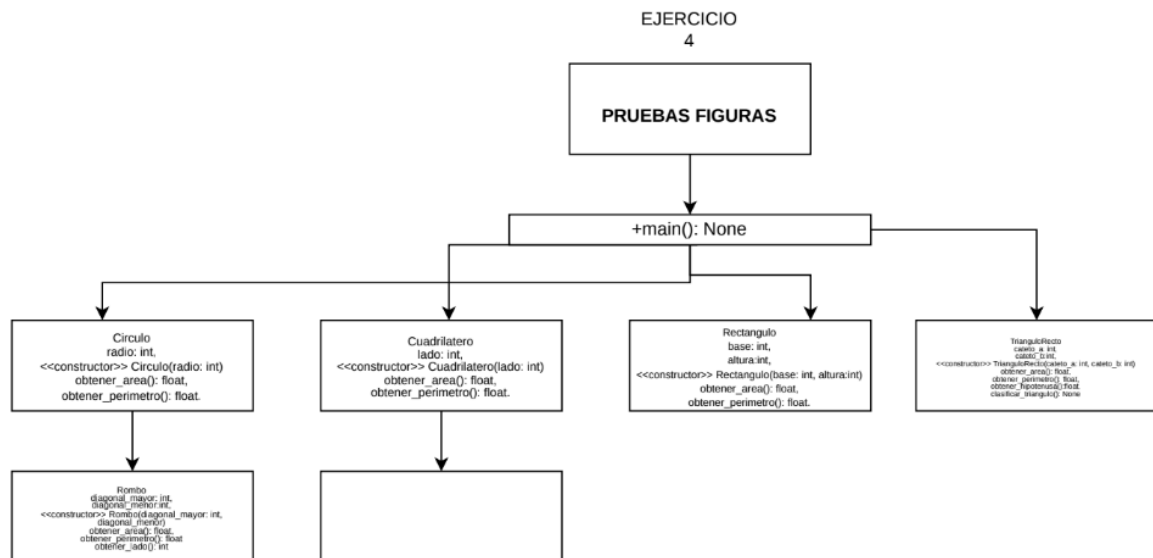
```

```

if __name__ == "__main__":
    ejecutar()

```

Ejercicio #2.4



Código fuente:

```

import math
class Circunferencia:
    def __init__(self, radio: int):
        self.radio = radio

```

```

def obtener_area(self):
    return math.pi * self.radio ** 2
def obtener_perimetro(self):
    return 2 * math.pi * self.radio
class Cuadrilatero:
    def __init__(self, lado: int):
        self.lado = lado
    def obtener_area(self):
        return float(self.lado ** 2)
    def obtener_perimetro(self):
        return float(4 * self.lado)

class Rectangulo:
    def __init__(self, ancho: int, alto: int):
        self.ancho = ancho
        self.alto = alto

    def obtener_area(self):
        return float(self.ancho * self.alto)

    def obtener_perimetro(self):
        return float(2 * (self.ancho + self.alto))

class TrianguloRecto:
    def __init__(self, cateto_a: int, cateto_b: int):
        self.cateto_a = cateto_a
        self.cateto_b = cateto_b

    def obtener_area(self):
        return self.cateto_a * self.cateto_b / 2

    def obtener_hipotenusa(self):
        return math.hypot(self.cateto_a, self.cateto_b)

    def obtener_perimetro(self):
        return self.cateto_a + self.cateto_b + self.obtener_hipotenusa()

    def clasificar_triangulo(self):
        h = self.obtener_hipotenusa()
        lados = [self.cateto_a, self.cateto_b, h]
        iguales = len({round(l, 9) for l in lados})
        if iguales == 1:
            print("Es un triángulo equilátero")
        elif iguales == 2:
            print("Es un triángulo isósceles")
        else:
            print("Es un triángulo escaleno")

```

```

class Rombo:
    def __init__(self, diagonal_mayor: int, diagonal_menor: int):
        self.diagonal_mayor = diagonal_mayor
        self.diagonal_menor = diagonal_menor

    def obtener_lado(self):
        return math.hypot(self.diagonal_mayor / 2, self.diagonal_menor / 2)

    def obtener_area(self):
        return self.diagonal_mayor * self.diagonal_menor / 2

    def obtener_perimetro(self):
        return 4 * self.obtener_lado()

class Trapecio:
    def __init__(self, base_mayor: int, base_menor: int, altura: int,
        lado_izquierdo: int, lado_derecho: int):
        self.base_mayor = base_mayor
        self.base_menor = base_menor
        self.altura = altura
        self.lado_izquierdo = lado_izquierdo
        self.lado_derecho = lado_derecho

    def obtener_area(self):
        return (self.base_mayor + self.base_menor) * self.altura / 2

    def obtener_perimetro(self):
        return float(self.base_mayor + self.base_menor
            + self.lado_izquierdo + self.lado_derecho)

class DemostracionFiguras:
    @staticmethod
    def ejecutar():
        figura1 = Circunferencia(2)
        figura2 = Rectangulo(1, 2)
        figura3 = Cuadrilatero(3)
        figura4 = TrianguloRecto(3, 5)
        figura5 = Rombo(8, 6)
        figura6 = Trapecio(10, 6, 4, 5, 5)

        print(f"El área del círculo es = {figura1.obtener_area()}")
        print(f"El perímetro del círculo es = {figura1.obtener_perimetro()}")
        print()
        print(f"El área del rectángulo es = {figura2.obtener_area()}")
        print(f"El perímetro del rectángulo es = {figura2.obtener_perimetro()}")
        print()
        print(f"El área del cuadrado es = {figura3.obtener_area()}")
        print(f"El perímetro del cuadrado es = {figura3.obtener_perimetro()}")
        print()
        print(f"El área del triángulo es = {figura4.obtener_area()}")

```

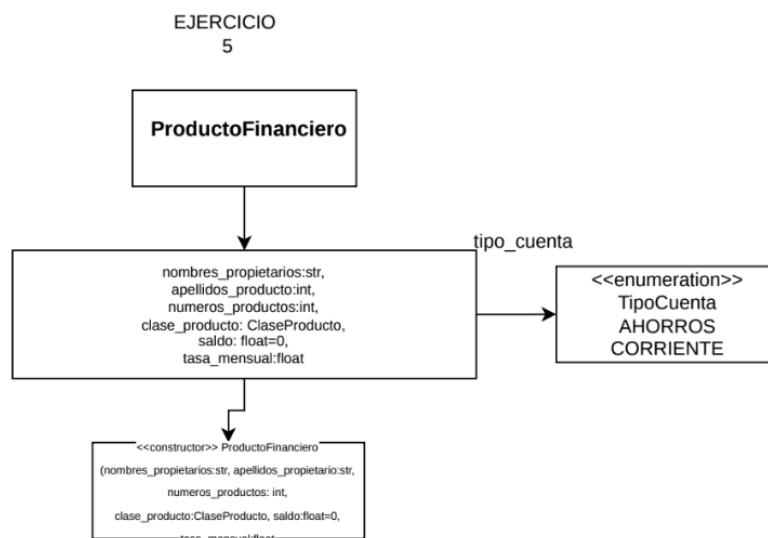
```

print(f"El perímetro del triángulo es = {figura4.obtener_perimetro()}")
figura4.clasificar_triangulo()
print()
print(f"El área del rombo es = {figura5.obtener_area()}")
print(f"El perímetro del rombo es = {figura5.obtener_perimetro()}")
print()
print(f"El área del trapecio es = {figura6.obtener_area()}")
print(f"El perímetro del trapecio es = {figura6.obtener_perimetro()}")

if __name__ == "__main__":
    DemostracionFiguras.ejecutar()

```

Ejercicios #2.5



Código fuente:

```

ifrom enum import Enum

class ClaseProducto(Enum):
    AHORROS = "AHORROS"
    CORRIENTE = "CORRIENTE"

class ProductoFinanciero:
    def __init__(self, nombres_propietario: str, apellidos_propietario: str,
        numero_producto: int, clase_producto: ClaseProducto,
        tasa_mensual: float = 0.0):
        self.nombres_propietario = nombres_propietario
        self.apellidos_propietario = apellidos_propietario
        self.numero_producto = numero_producto
        self.clase_producto = clase_producto
        self.disponible = 0.0
        self.tasa_mensual = tasa_mensual

```

```

def mostrar(self):
    print(f"Nombres del titular = {self.nombres_propietario}")
    print(f"Apellidos del titular = {self.apellidos_propietario}")
    print(f"Número de cuenta = {self.numero_producto}")
    print(f"Tipo de cuenta = {self.clase_producto.name}")
    print(f"Saldo = {self.disponible}")
    print(f"Interés mensual = {self.tasa_mensual} %")

def ver_disponible(self):
    print(f"El saldo actual de la cuenta {self.numero_producto} es ${self.disponible}")

def depositar(self, monto: int):
    if monto <= 0:
        print("El valor a consignar debe ser mayor que cero.")
        return False
    self.disponible += monto
    print(f"Se ha consignado ${monto} en la cuenta. El nuevo saldo es ${self.disponible}")
    return True

def extraer(self, monto: int):
    if monto <= 0:
        print("El valor a retirar debe ser mayor que cero.")
        return False
    if monto > self.disponible:
        print("Saldo insuficiente para realizar el retiro.")
        return False
    self.disponible -= monto
    print(f"Se ha retirado ${monto} en la cuenta. El nuevo saldo es ${self.disponible}")
    return True

def comparar_con(self, producto: "ProductoFinanciero"):
    if self.disponible > producto.disponible:
        print(f"La cuenta {self.numero_producto} tiene mayor saldo que la cuenta {producto.numero_producto}.")
    elif self.disponible < producto.disponible:
        print(f"La cuenta {producto.numero_producto} tiene mayor saldo que la cuenta {self.numero_producto}.")
    else:
        print("Ambas cuentas tienen el mismo saldo.")

def transferir_a(self, producto: "ProductoFinanciero", monto: int):
    if monto <= 0 or monto > self.disponible:
        print("No se pudo realizar la transferencia.")
        return False
    self.disponible -= monto
    producto.disponible += monto
    print(f"Se han transferido ${monto} de la cuenta {self.numero_producto} "
          f"a la cuenta {producto.numero_producto}.")
    return True

def aplicar_interes(self):
    self.disponible += self.disponible * self.tasa_mensual / 100
    print(f"Se aplicó un interés del {self.tasa_mensual} %. El nuevo saldo es ${self.disponible}")
    return self.disponible

def ejecutar():
    producto = ProductoFinanciero("Pedro", "Pérez", 123456789, ClaseProducto.AHORROS, 1.5)
    producto.mostrar()
    producto.depositar(200000)

```

```
producto.depositar(300000)
producto.extraer(400000)
producto.aplicar_interes()

print()
otro = ProductoFinanciero("Luis", "León", 987654321, ClaseProducto.CORRIENTE)
otro.depositar(50000)
producto.comparar_con(otro)
producto.transferir_a(otro, 60000)
producto.ver_disponible()
otro.ver_disponible()

if __name__ == "__main__":
    ejecutar()
```

URL: <https://github.com/karistiqqq/POO-2026>